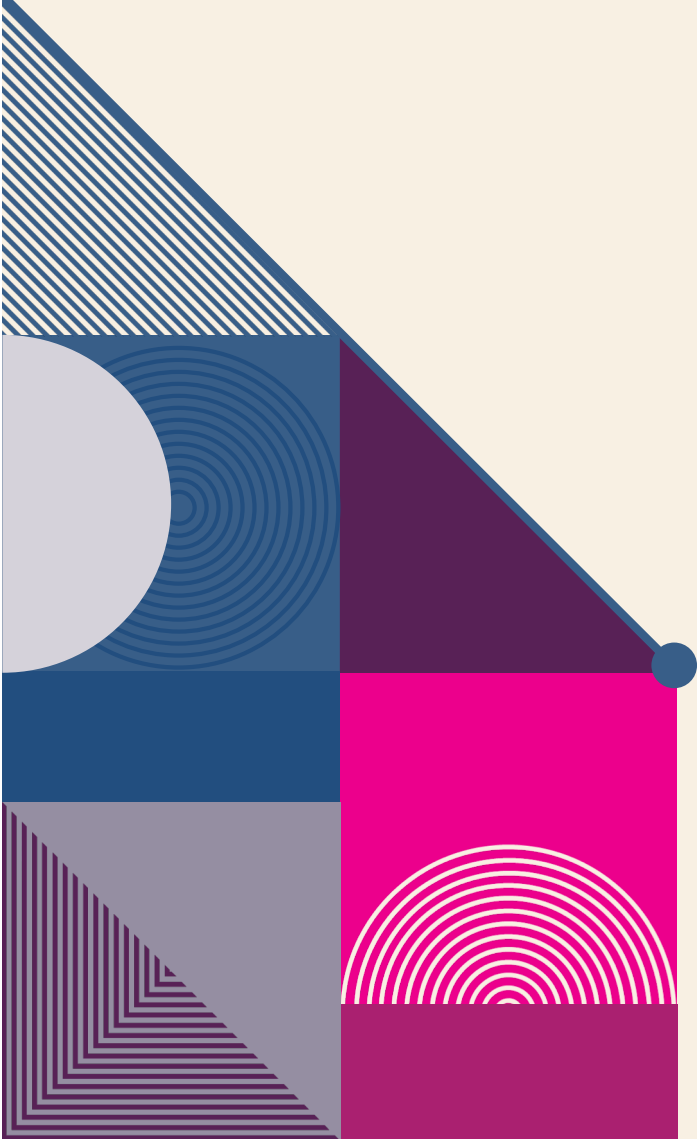


A decorative geometric pattern in the top-left corner of the slide. It features a dark blue background with a white circle at the top-left corner. Below the circle, there are concentric circles in a lighter blue shade. To the right of the circles, there are diagonal lines in a magenta color. Below the circles, there are concentric squares in a magenta color. To the right of the squares, there are diagonal lines in a magenta color. The pattern is composed of various geometric shapes and colors, including dark blue, magenta, and white.

Section 3.3 – Redshift Features



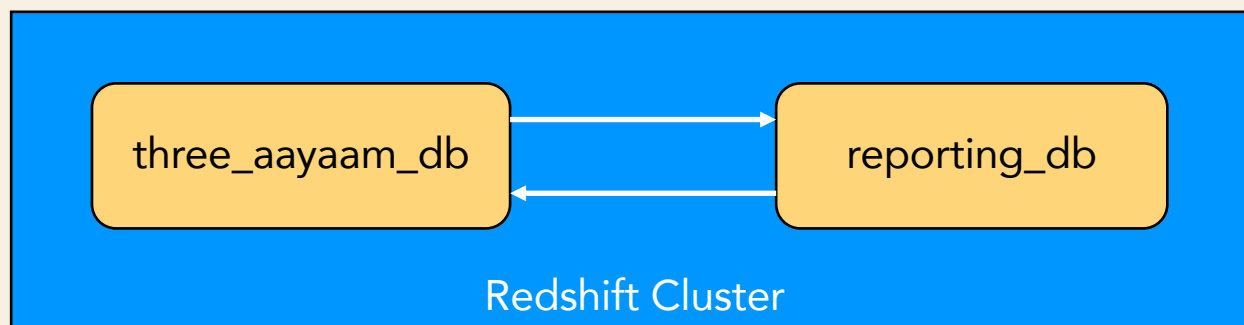
Agenda

- Materialized Views
- Federated Queries
- Redshift Spectrum
- Streaming Ingestion into Redshift
- Data Sharing
- Machine Learning (Theory Only)
- Use Cases (Deep dive, Features)

The image features a low-angle photograph of modern skyscrapers with glass facades, set against a clear blue sky. This photograph occupies the left portion of the frame. Overlaid on the bottom-left corner of the image is a series of overlapping geometric shapes: a red triangle with white diagonal lines, a light gray semi-circle, a dark blue square with concentric circles, a dark purple triangle, and a bright pink square. A white dot is positioned at the intersection of the purple triangle and the pink square. The right half of the image is a solid, medium-blue background.

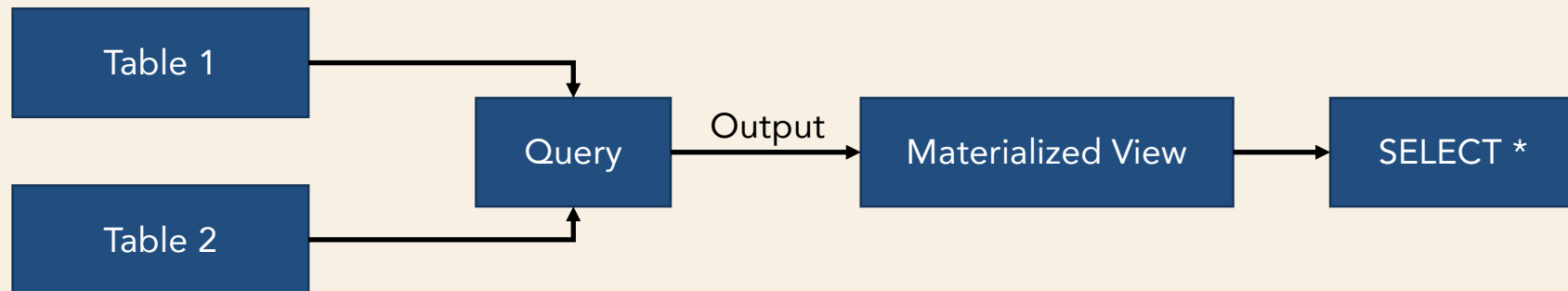
Materialized View

Query Across Databases



- Query tables across databases in the same cluster.
- Only supported in RA3 node types.
- Querying tables with interleaved sort keys are not supported.
- `database_name.schema_name.table_name`
`database_name.schema_name.table_name.column_name`

Materialized View



- Stores the result of a query in storage.
- Improve performance
- REFRESH MATERIALIZED VIEW
- Automatic refresh

Materialized View (MV)

- Stores the pre-computed result of a query in storage.
- Improve performance
- REFRESH MATERIALIZED VIEW
- Automatic refresh
- STV_MV_INFO
- STL_MV_STATE
- SVL_MV_REFRESH_STATUS
- STV_MV_DEPS

Materialized View (MV)

```
CREATE MATERIALIZED VIEW <mv-name>  
DISTKEY (col, ...) DISTSTYLE EVEN | ALL | KEY  
SORTKEY (col, ...)  
AUTO REFRESH YES | NO  
AS <query>
```

Redshift auto refreshes
MVs as soon as
possible after base
tables changes.

- <query> can contain EXTERNAL tables, Federated tables and other MVs.
- Columns to the base table can be added.
- Dropping, changing data types, renaming columns will need the MV to be dropped and re-created for refresh to work.
- ORDER BY, LIMIT, OFFSET cannot be used in the definition of MV.
- Views, System tables and views cannot be there in the definition of MV.
- Only AUTO ANALYZE, cannot be run manually.

Materialized View (MV)

```
ALTER MATERIALIZED VIEW <mv-name>  
  AUTO REFRESH YES | NO  
  ROW LEVEL SECURITY ON | OFF  
  
REFRESH MATERIALIZED VIEW <mv-name>
```

- Full refresh.
- Incremental refresh – **Supports** SELECT, INNER JOIN, GROUP BY, HAVING, WHERE, COUNT, SUM, MIN, MAX, AVG.
- Increment refresh is **NOT** supported with the following.
 - Outer join (Right, Left and Full).
 - Aggregate functions (except COUNT, SUM, MIN, MAX and AVG).
 - DISTINCT, Window functions, Subqueries.

Automated Materialized View

- Creates MVs automatically based on activity and performance of the database.
- Monitors workload continuously using ML and creates new MVs when they are beneficial.
- Automatic incremental refresh.



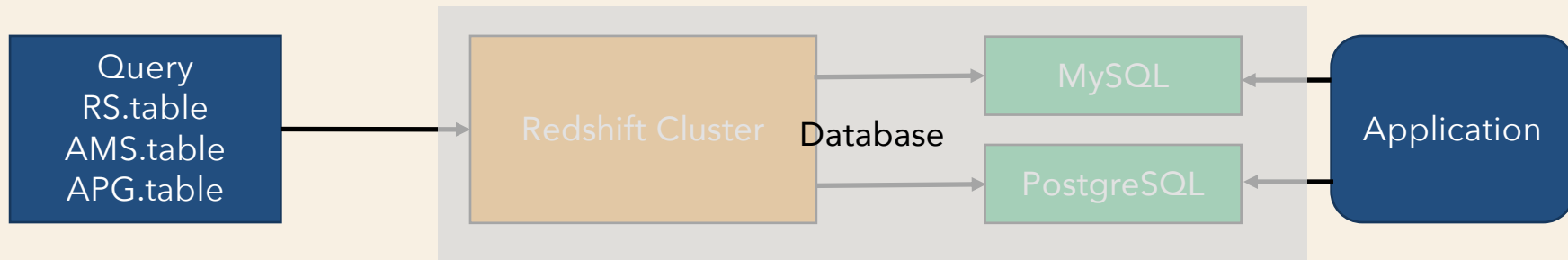
Hands-on

- Experiment with query result caching. Set it to OFF.
- Load orders, customer and sellers table with 1 file.
- Create multiple MVs using both manual and auto refresh option.
- Execute queries on MVs.
- Load the tables again with 2nd set of files.
- Check MV refresh status using System tables.
- Manually refresh some of the MVs and ALTER them.
- Execute MV query again.



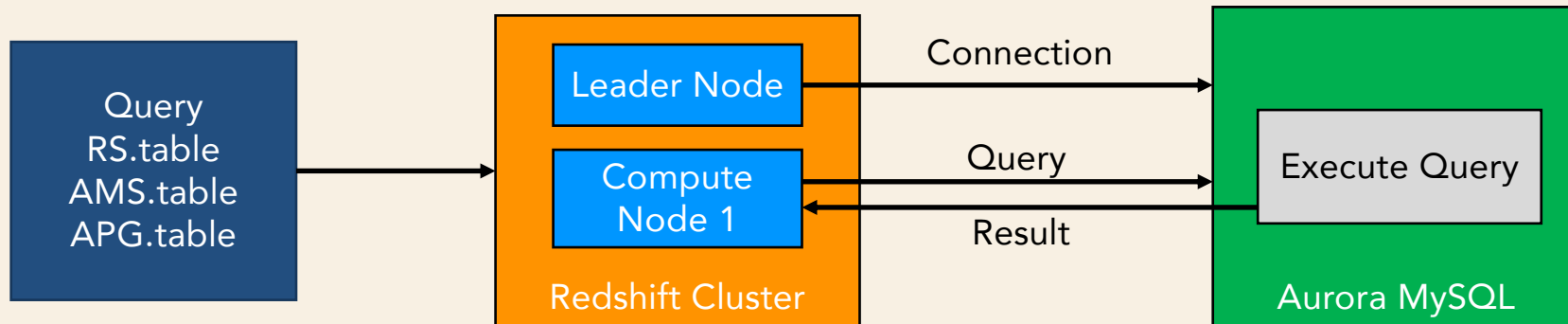
Query Federation

Redshift Federation



- Brings multiple databases at one place from the user perspective.
- Redshift supports querying RDS/Aurora MySQL and PostgreSQL from the cluster.
- `SVL_FEDERATED_QUERY`

Redshift Federation



- External sources can ONLY BE READ, no Write is allowed.
- Federated queries are NOT allowed from RDS or Aurora to Redshift.
- External schema has to be dropped and re-created. ALTER SCHEMA is not supported.

Hands-on

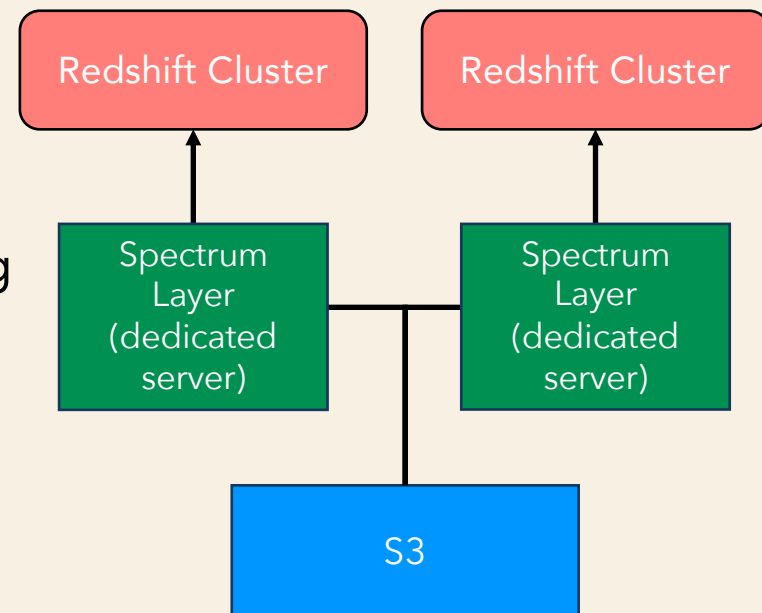
- What is AWS Secret Manager? Create a Secret.
- Create an Aurora MySQL cluster.
- Create customer table and load data from S3.
- Create EXTERNAL SCHEMA (from Redshift) for Aurora MySQL.
- Join "orders" table in Redshift cluster with "customers" table in Aurora MySQL from Redshift.

An abstract graphic design featuring a low-angle photograph of modern skyscrapers with glass facades, reflecting the sky. The image is partially obscured by a diagonal white line that separates a dark blue background on the right from a collage of geometric shapes on the left. These shapes include a red triangle with white diagonal lines, a light gray semi-circle, a dark blue square with concentric circles, a dark purple triangle, and a bright pink square. A small white circle is located at the intersection of the diagonal line and the pink square.

Redshift Spectrum

Spectrum – Read from S3

- Read data from files in S3.
- Don't need to load data into Redshift.
- Predicate Filtering, Aggregation.
- Need a Redshift cluster up and running for Spectrum to work.
- UPDATE and DELETE are not allowed.
- SELECT and INSERT are allowed.
- Combine data on Redshift, Aurora/RDS (MySQL and PostgreSQL) and S3



Spectrum – External Schema

- References external database in external metastore or data catalog.
- External data catalog - Athena, Glue Data Catalog, Redshift, Hive Metastore.
- Should have permission to access metastore.

```
CREATE EXTERNAL SCHEMA <schema>  
FROM DATA CATALOG | HIVE METASTORE | POSTGRES | MYSQL | REDSHIFT  
IAM_ROLE <iam-role>  
DATABASE <external-database-name>  
CREATE EXTERNAL DATABASE IF NOT EXISTS
```

Spectrum – External Table

```
CREATE EXTERNAL TABLE <ext_schema.table> (column data_type)
PARTITIONED BY (col_name data_type)
ROW FORMAT DELIMITED LINES TERMINATED BY '\n' FIELDS TERMINATED BY ','
ROW FORMAT SERDE 'serde_name'
STORED AS PARQUET | ORC | AVRO | TEXTFILE
LOCATION '<s3://bucket/folder/>' | '<s3://bucket/manifest_file>'
TABLE PROPERTIES ( 'property_name'='property_value' )
```

- SVL_S3QUERY_SUMMARY

Spectrum – Supported File Formats

File format	Columnar	Supports parallel reads	Split unit
Parquet	Yes	Yes	Row group
ORC	Yes	Yes	Stripe
RCFile	Yes	Yes	Row group
TextFile	No	Yes	Row
SequenceFile	No	Yes	Row or block
RegexSerde	No	Yes	Row
OpenCSV	No	Yes	Row
AVRO	No	Yes	Block
Ion	No	No	N/A
JSON	No	No	N/A

Compression Algorithm	File Extension	Supports Parallel Reads
Gzip	.gz	No
Bzip2	.bz2	Yes
Snappy	.snappy	No

Hands-on

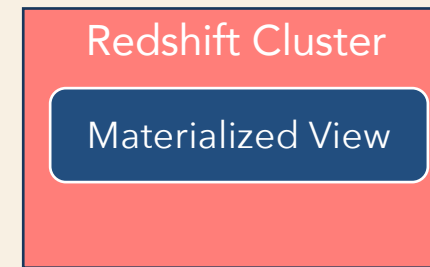
- Create an EXTERNAL SCHEMA in Redshift and Glue Catalog.
- Create EXTERNAL TABLE that points to sellers file in S3.
- Query the records.
- Join orders table in Redshift, customers table in Aurora MySQL and sellers table in S3



Streaming Ingestion

What is Streaming Data

- Continuously incoming unbounded data – Low latency, High speed ingestion.
- Ingestion needs storage
 - Kafka (MSK)
 - Kinesis
- Near-real time analytics in Redshift using MV.



MV for Streaming Ingestion

- MV should be incrementally refreshable.
- No JOINS within the MV.
- Auto refresh loads data from stream as it arrives.
- Remove records using TRUNCATE and DELETE.
- Move records from Streaming MV to a Table.

```
ALTER TABLE <table-name> APPEND FROM <streaming-mv>  
IGNOREEXTRA | FILLTARGET
```

Streaming MV from Kinesis

```
CREATE EXTERNAL SCHEMA <kinesis-schema>  
FROM KINESIS  
IAM_ROLE <iam-role>;
```

```
CREATE MATERIALIZED VIEW <kinesis-mv>  
AUTO REFRESH YES AS  
SELECT *  
FROM <kinesis-schema>.<kds-stream-name>;  
  
REFRESH MATERIALIZED VIEW <mv-name>;
```

Metadata column	Data type	Description
approximate_arrival_timestamp	timestamp without time zone	The approximate time that the record was inserted into the Kinesis stream
partition_key	varchar(256)	The key used by Kinesis to assign the record to a shard
shard_id	char(20)	The unique identifier of the shard within the stream from which the record was retrieved
sequence_number	varchar(128)	The unique identifier of the record from the Kinesis shard
refresh_time	timestamp without time zone	The time the refresh started
kinesis_data	varbyte	The record from the Kinesis stream

Streaming MV from Kafka (MSK)

```
CREATE EXTERNAL SCHEMA <msk-schema>
FROM MSK
IAM_ROLE <iam-role>
AUTHENTICATION none | iam
URI <msk-cluster-uri>
CLUSTER_ARN <msk-cluster-arn>;
```

```
CREATE MATERIALIZED VIEW <kafka-mv>
AUTO REFRESH YES AS
SELECT *
FROM <kafka-mv>."<kafka-topic>";

REFRESH MATERIALIZED VIEW <kafka-mv>;
```

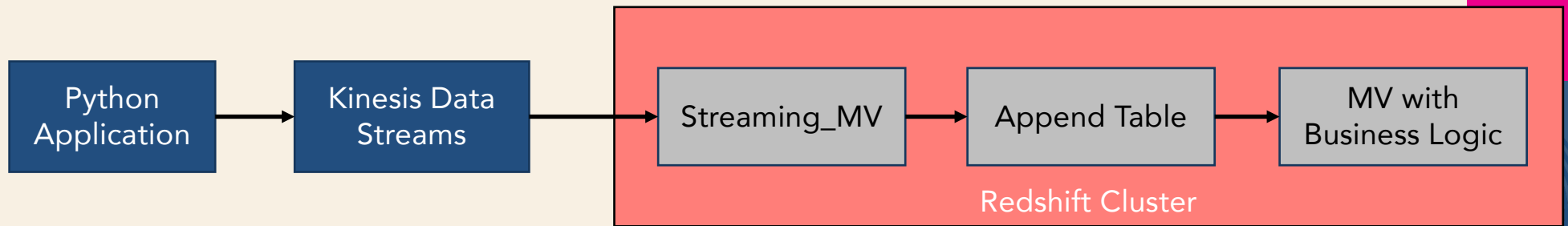
Streaming MV from Kafka (MSK)

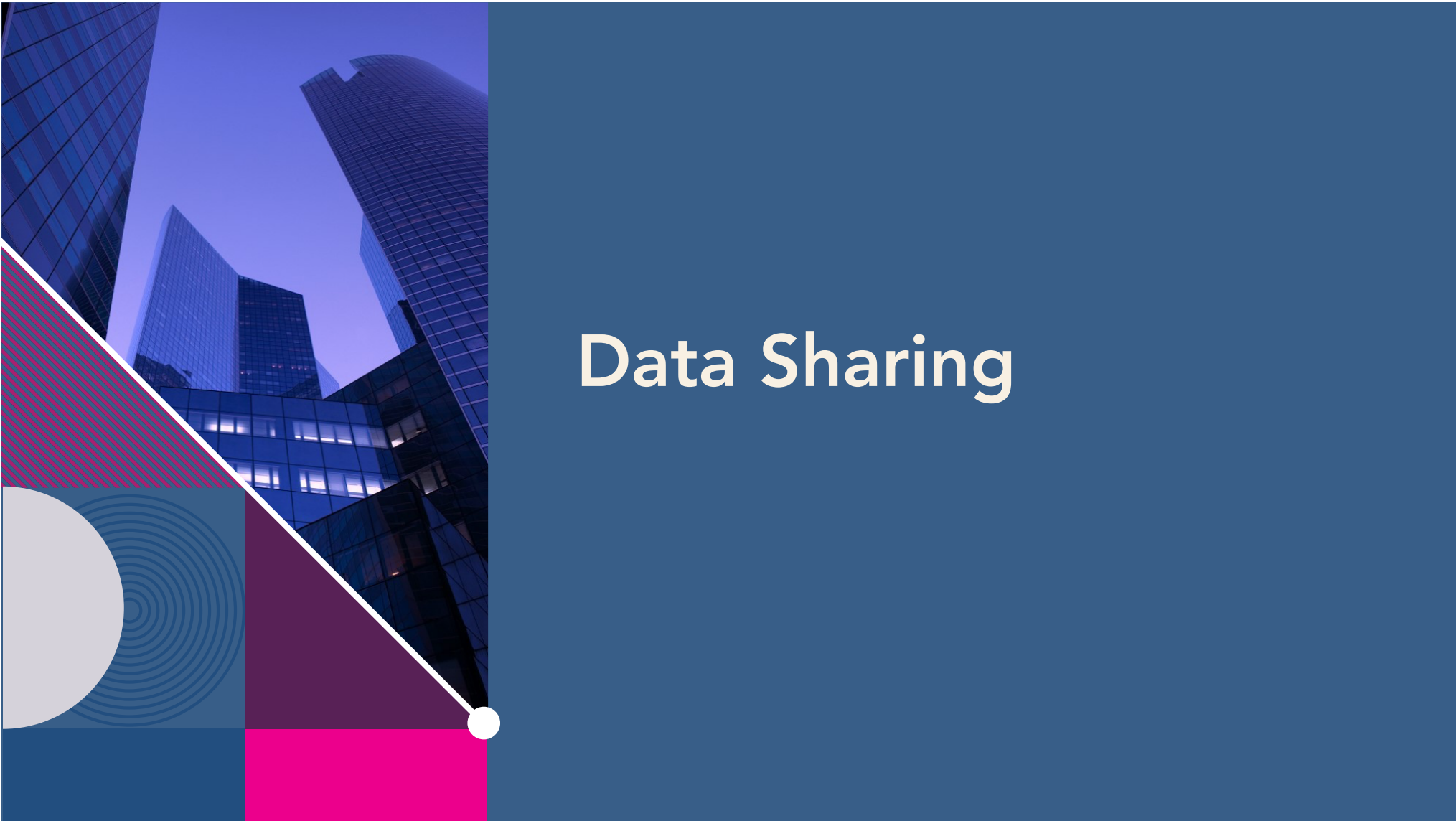
Metadata column	Data type	Description
kafka_partition	bigint	Partition id of the record from the Kafka topic
kafka_offset	bigint	Offset of the record in the Kafka topic for a given partition
kafka_timestamp_type	char(1)	Type of timestamp used in the Kafka record: • <i>C</i> – Record creation time (CREATE_TIME) on the client side • <i>L</i> – Record append time (LOG_APPEND_TIME) on the Kafka server side • <i>U</i> – Record creation time is not available (NO_TIMESTAMP_TYPE)
kafka_timestamp	timestamp without time zone	The timestamp value for the record
kafka_key	varbyte	The key of the Kafka record
kafka_value	varbyte	The record received from Kafka
kafka_headers	super	The header of the record received from Kafka
refresh_time	timestamp without time zone	The time the refresh started

Hands-on

- Create a Kinesis Data Stream.
- Create External Schema.
- Create Materialized View.
- Create table to APPEND data and an MV on top of it.
- Execute REFRESH command.
- Run Python application to load data to Kinesis.
- Check data in MV.
- Append to the table and perform transformation.

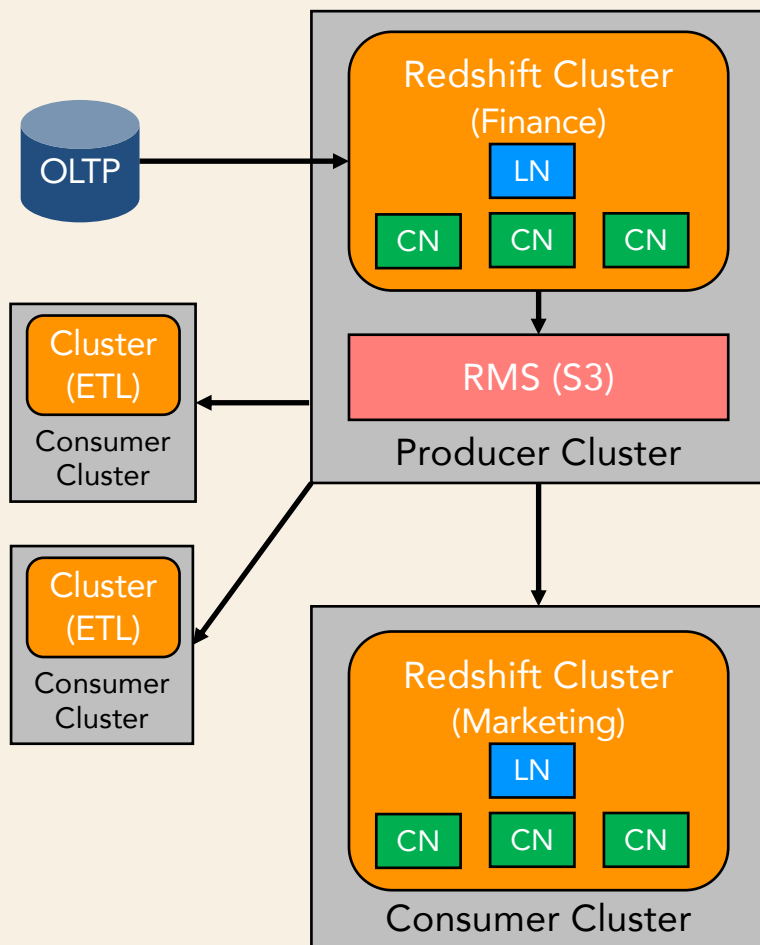
Hands-on





Data Sharing

Sharing Data across Clusters



- Share data of one Redshift cluster with another cluster
 - Across AWS accounts
 - Across regions
 - Between provisioned and serverless.
- Individual objects (databases, schemas, tables, views, MVs) can be shared for reads and writes (preview).
- Can't have external schemas, tables, tables with interleaved sort key.
- Consumers cannot share to another datashare.

Creating Datashare

- `CREATE DATASHARE <datashare-name>;`
- `ALTER DATASHARE <datashare-name> ADD SCHEMA <schema1>;`
- `ALTER DATASHARE <datashare-name> ADD TABLE <table1>;`
- `ALTER DATASHARE <datashare-name> ADD ALL TABLES IN SCHEMA <schema2>;`
- `ALTER DATASHARE <datashare-name> SET INCLUDENEW = TRUE FOR SCHEMA <schema>;`
- `SHOW DATASHARES;`
- `GRANT USAGE ON DATASHARE <datashare-name> TO NAMESPACE '<redshift-namespace>';`
- `SVV_DATASHARES, SVV_DATASHARE_CONSUMERS, SVV_DATASHARE_OBJECTS.`

Hands-on

- Create a new cluster with RA3 node type.
- Create datashare.
- Add retail.orders table in the datashare.
- Add three_aayaam_oltp in the datashare.
- Add schema public in the datashare.
- Share the datashare with the new cluster.
- Check the objects from the new cluster.





Machine Learning

Redshift ML

- Why Machine Learning
- Model, Prediction
- Training data, Testing data and Validation data
- Redshift ML can Create, Train and Deploy ML models using SQL commands

```
CREATE MODEL <model-name> FROM <table-name> | <SELECT query>  
MODEL TYPE XGBOOST | MLP | LINEAR_LEARNER  
PROBLEM_TYPE REGRESSION | BINARY_CLASSIFICATION | MULTICLASS_CLASSIFICATION
```



THANK YOU

www.3aayaam.in